Consultas — Etapa 5 (generación de código)

Agustín Paul — L.U. 147524

Te paso las dudas de la Etapa 5. En cada una hay un ejemplo y lo que pienso hacer; con un «sí» o con la otra opción me alcanza. «El documento» es *Semántica de MiniJava – Documento completo* (v3.0) y «el apunte», el *Apunte CeIVM 2026*. Las líneas arrancan en 1.

1. ¿Qué error doy si no hay `main`, o si hay más de uno?

El documento (§8 y §14) deja para esta etapa el control del punto de entrada, y el apunte (§15) dice que todo programa declara un `static void main()`. No encontré qué reportar:

```
1 class Aa {
2     void main() { }
3 }
```

Mi idea: cuenta sólo un `main` estático, `void` y sin parámetros, declarado en la clase (no heredado). Si no hay ninguno, un error semántico en el fin de archivo, como hace el sintáctico cuando falta algo al final: acá, `[Error:$|4]`, porque el archivo termina con un salto de línea. Si hay dos, en clases distintas, el error va en el nombre del segundo, como un nombre repetido.
Alternativas: reportar en otro token, o no controlarlo (en Java compila y falla recién al ejecutar).

2. Los errores de ejecución que controla mi código, ¿con qué mensaje terminan?

```
1 class Init {
2     static void main() {
3         var a = new int[2];
4         a[5] = 1;
5     }
6 }
```

El apunte (§12 y §17) pide controlar el índice fuera de rango y el tamaño negativo, y sugiere una rutina que imprima un mensaje y haga `HALT`. Pero tras un `HALT` la CeIVM imprime «La ejecución del programa finalizó exitosamente.», como en un programa sin errores. Mi idea: imprimir `Error en tiempo de ejecucion: indice fuera de rango` y terminar con `HALT`.
¿Está bien así, o preferís que termine provocando un error de la propia VM, para que no diga «exitosamente»? ¿Hay un formato de mensaje que esperen?

3. ¿Qué valor tienen los escapes en los literales?

```
class Init {
    static void main() {
        System.printCln('\n');
        System.printCln('\q');
        System.printSln("a\tb");
    }
}
```

Los aspectos léxicos (§2.3.2) aceptan `'\x'` con cualquier `x`, pero no dicen su valor, y para los Strings dicen que «el valor del literal corresponde a la cadena de caracteres entre las comillas». Mi idea: como en Java, `\n` es el salto de línea, `\t` el tabulador, etcétera, también dentro de los Strings; y como Java rechaza `'\q'`, para los escapes que Java no tiene, el valor es el carácter mismo (`q`).
Alternativa: los Strings se toman crudos, y `"a\tb"` tiene cuatro caracteres.

4. Para confirmar: en una asignación a un atributo o a un arreglo, ¿evalúo primero el lado derecho?

```
class Init {
    int x;
    static Init obj() { debugPrint(1); return new Init(); }
    static int val() { debugPrint(2); return 5; }
    static void main() {
        obj().x = val();
    }
}
```

Sigo el esquema de la Clase 16 (slides 18 a 29) y del apunte (§7): primero el valor, después la referencia, `SWAP` y `STOREREF`. Imprime 2 y después 1. En Java imprime 1 y después 2: el lado izquierdo se evalúa primero.
Alternativa: hacerlo como Java.

5. Para confirmar: dos literales String iguales, ¿son el mismo objeto?

```
class Init {
    static void main() {
        System.printBln("hola" == "hola");
    }
}
```

El apunte (§13) aloja cada literal en el heap y el documento (§11.2) compara las referencias por identidad, así que, creando el String cada vez que se evalúa el literal, imprime `false`. En Java imprime `true`, porque los literales iguales son el mismo objeto.
Alternativa: crear cada literal distinto una sola vez, al arrancar, para que dé `true` como en Java.